these methods require additional training using images with localized objects as ground truth. Finally, building on top of diffusion, some recent works show controllable image generation from various conditioning signals [1, 48, 16].

Diffusion-Based Image Editing. Most aforementioned methods lack the ability to control or edit an already generated image, or even the ability to edit real images. For example, simply changing a word in the original prompt typically leads to a drastically different generation. This can be circumvented by providing or generating masks for the objects of interest [24, 2]. Prompt-to-prompt [12] addresses this issue with simple text-based edits by exploiting the fact that the cross-attention layers present in most state-of-the-art architectures connect word tokens to the spatial layout of the generated images. Text-based image editing can also be achieved through single-image model fine-tuning [19, 39]. However, these approaches, while successful at semantically editing entities can only apply such edits *in-place* and do not allow editing of the spatial layout itself.

3. Method

We consider the problem of layout-guided text-to-image generation. Text-based image generators allow to sample images $\boldsymbol{x} \in \mathbb{R}^{3\times H\times W}$ from a conditional distribution $p(\boldsymbol{x} \mid y)$ where $y \in \Sigma^*$ is a language description, Σ is a vocabulary (a list of words), and Σ^* is the set of all possible sequences of words. Given one such generator off-the-shelf, we wish to steer the output to match a desired layout for the generated composition, *without further training*. In other words, our objective is to investigate whether pre-trained text-to-image generators can adhere to a layout specified by the user during inference, without having been trained with explicit layout conditioning. In the simplest case, given the textual description y, the index i of a word y_i in the text prompt, and a bounding box B, we would like to generate an image $\boldsymbol{x}$ that contains y_i *inside* B, essentially modifying the generator to sample from a new distribution $p(\boldsymbol{x} \mid y, B, i)$ with additional controls.

The rest of the section is structured as follows. In Section 3.1 we briefly review several technical details of Stable Diffusion [31], as a publicly accessible, state-of-the-art text-to-image generator. In Section 3.2, we introduce a method that achieves layout guidance during the generation process by intervening in the way the underlying image generator associates visual and textual information. This is achieved via manipulation of the attention response in certain cross-attention layers in the architecture, as well as via an analysis of how textual information is encoded and how the latter affects the output. Finally, in Section 3.3 we discuss how to make this approach applicable to editing real images.

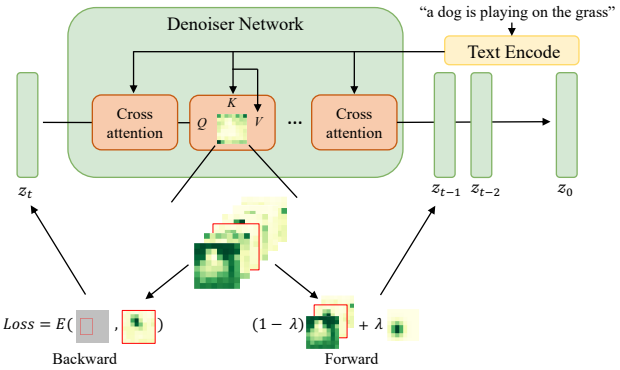


Figure 2: An overview of the two layout guidance strategies: The cross-attention map for a chosen word token is marked with a red border. In forward guidance, we bias (spatially) the word token, start token, and padding tokens. In backward guidance, we perform backpropagation during the inference process to optimize the latent.

3.1. Preliminaries: Stable Diffusion

We consider Stable Diffusion (SD) as a representative conditional image generator of an important class of recent image generators based on diffusion [29, 34, 31]. The SD model comprises a *textual encoder* $Y = \phi(y)$ that maps the input prompt into a tensor of fixed dimension $Y \in \mathbb{R}^{M\times N}$. This works by prepending a start (`[SoT]`) symbol to y and appending $N-|y|-1$ padding (`[EoT]`) symbols at the end, to obtain N symbols in total. Then, the function ϕ, implemented as a large language model (LLM), takes the padded sequence of words as input and produces a corresponding sequence of token vectors $Y_i \in \mathbb{R}^M$ with $i \in \{1, \dots, N\}$ as output.

While not crucial for our discussion, the SD model also contains an encoding network h that maps images $\boldsymbol{x}$ to corresponding latent codes $\boldsymbol{z} = h(\boldsymbol{x}) \in \mathbb{R}^{4\times \frac{H}{s}\times \frac{W}{s}}$, where s divides H and W. The function h is an autoencoder and has a left inverse h^*, such that $\boldsymbol{x} = h^* \circ h(\boldsymbol{x})$. The main purpose of this component is to replace the problem of modeling $p(\boldsymbol{x} \mid y)$ with the problem of modeling $p(\boldsymbol{z} \mid y)$, reducing the spatial resolution s-fold.

The other key component of SD is an iterative conditional denoising network D. This network is trained to output a conditional sample $\boldsymbol{z} \sim p(\boldsymbol{z} \mid y)$ of the latent code $\boldsymbol{z}$. It is designed to take a noised sample $\boldsymbol{z}_t = \alpha_t \boldsymbol{z} + \sqrt{1-\alpha_t}\boldsymbol{\epsilon}_t$, where $\boldsymbol{\epsilon}_t$ is normally distributed noise and α_t is a decreasing sequence, from $\alpha_0 \approx 1$ to $\alpha_T \approx 0$, representing the noise schedule. Then, the network D returns an estimate of the noised sample $\boldsymbol{z}_t$: $D(\boldsymbol{z}_t, y, t) \approx \boldsymbol{\epsilon}_t$. To sample an image, one first samples $\boldsymbol{z}_T$, which is normally distributed, and applies D iteratively, to obtain the intermediate codes $\boldsymbol{z}_{T-1}, \dots, \boldsymbol{z}_1, \boldsymbol{z}_0 \approx \boldsymbol{z}$. Finally, $\boldsymbol{z}$ is converted back to an image via the image decoder $\boldsymbol{x} = h^*(\boldsymbol{z})$.